

IoT-Enabled Dual-Stage PCB Defect Detection System: A Hybrid Approach Using YOLOv11

Student Name: PADMA MUTHU LAKSHMANAN

Faculty Advisor: Dr. Ezhilarasi

Department: Instrumentation and Control Engineering

Year: 2025-2026

Abstract

The rapid transition of the electronics manufacturing and semiconductor industry toward Industry 4.0 standards has required the development of intelligent, real-time Quality Assurance (QA) systems. Traditional inspection methodologies, relying on manual visual checks or rigid Automated Optical Inspection (AOI) hardware, suffer from significant limitations in flexibility, speed, and predictive capability. This project proposes a cost-effective, end-to-end automated inspection system that strategically decouples the defect detection process into two specialized pipelines: Model A, optimized for high-precision bare-board fabrication, and Model B, tailored for high-speed component assembly verification. Leveraging the YOLOv11 single-stage detector architecture trained on a hybrid dataset of 1,698 images, the system mitigates the domain gap between synthetic benchmarks and real-world factory conditions. Empirical results demonstrate a Combined Average mAP@0.5 of 91.12% with 90.53% Recall across 10 defect classes, and an Average Inference Speed of 78.5 FPS, significantly outperforming traditional two-stage detectors. Furthermore, the integration of an Edge-to-Cloud telemetry pipeline via Microsoft Azure IoT Hub establishes a real-time "Digital Twin" of the manufacturing process, enabling predictive analytics and remote monitoring.

1. Introduction

1.1 Motivation and Problem Statement

As Printed Circuit Board (PCB) designs evolve toward higher density and miniaturization, the economic impact of manufacturing defects has escalated dramatically. A single "Open Circuit" in a bare board results in immediate material waste, while a "Solder Bridge" in a populated board can lead to catastrophic field failures and severe reputational damage. The industry faces a spectrum of failure modes, ranging from geometric fabrication errors like 'Mouse Bites' and 'Spurs' to complex assembly faults such as 'Cold Solder' joints and 'Component Misalignment,' each requiring distinct visual inspection strategies.

Current industrial solutions are polarized: factories must choose between expensive, hardware-centric AOI machines that lack adaptability, or manual inspection, which is prone to 20-30% error rates due to operator fatigue. There exists a critical need for a flexible, software-defined inspection system that combines the accuracy of deep learning with the connectivity of the Industrial Internet of Things (IIoT).

1.2. Literature Review

The evolution of Printed Circuit Board (PCB) defect detection has been marked by a transition from manual inspection to automated, data-driven methodologies. Early research primarily focused on reference comparison methods and morphological image processing. However, recent studies indicate that these traditional approaches are increasingly insufficient for modern High-Density

Interconnect (HDI) boards due to their inability to adapt to complex textures and variable industrial lighting conditions. Consequently, the focus of contemporary research has shifted almost exclusively toward deep learning-based solutions.

1.2.1 Evolution of Detection Architectures

Initial attempts to apply deep learning to PCB inspection utilized two-stage object detectors. Research published in *IEEE Access* demonstrated that region-proposal networks, such as Faster R-CNN, could achieve high detection accuracy (approx. 98%) by performing a preliminary scan for regions of interest before classification. While effective for precision-critical tasks, subsequent analysis highlighted a major drawback: these models typically suffer from high computational latency, rendering them unsuitable for high-speed Surface Mount Technology (SMT) production lines that require throughputs exceeding 50 frames per second.

To address the latency bottleneck, more recent work has pivoted toward single-stage detectors. Studies exploring the "You Only Look Once" (YOLO) architecture have shown that regression-based frameworks can balance speed and accuracy effectively. For example, research featured in *Nature Scientific Reports* validated a YOLO-based framework that achieved real-time inference speeds on edge hardware while maintaining detection rates comparable to two-stage models. Furthermore, the integration of advanced architectural features, such as multi-scale attention mechanisms and decoupled detection heads, has been shown to significantly improve the detection of small, localized defects like "spurs" and "mouse bites".

1.2.2 Challenges in Data and Class Imbalance

A persistent challenge identified across multiple studies is the issue of class imbalance. In a typical manufacturing environment, defective samples are rare compared to non-defective ones. Research investigating dataset composition found that models trained on imbalanced data tend to exhibit high bias, frequently failing to detect minority defect classes. To mitigate this, recent literature has proposed the use of data augmentation techniques—such as Mixup and Mosaic augmentation—to synthetically increase the representation of rare defects. Additionally, semi-supervised learning strategies have been introduced to leverage large volumes of unlabeled production data, thereby reducing the dependency on labor-intensive manual annotation.

1.2.3 Deployment and System Integration

While algorithmic performance has improved, the literature reveals a significant gap in system-level deployment. Most existing studies validate models in offline simulation environments, neglecting the practical constraints of "Edge-to-Cloud" integration. A comprehensive review on *ResearchGate* noted that while detection algorithms are mature, frameworks that successfully integrate edge inference with real-time cloud telemetry for "Digital Twin" applications remain scarce. Recent efforts have begun to address this by optimizing models for specific hardware accelerators (e.g., TensorRT, ONNX), but fully integrated, end-to-end industrial solutions are still limited in academic publications.

1.2.4 Summary of Research Gaps

Despite these advancements, current literature exhibits specific limitations. First, most studies rely exclusively on synthetic datasets that lack the textural complexity of real-world assembled boards. Second, there is a dichotomy in optimization objectives: researchers typically optimize for either precision (Fabrication) or speed (Assembly), rarely presenting a unified architecture that addresses both stages simultaneously. This project aims to bridge these specific gaps by proposing a

dual-stream architecture that optimizes each manufacturing stage independently while validating performance on a hybrid dataset of real-world and synthetic imagery.

2. Methodology

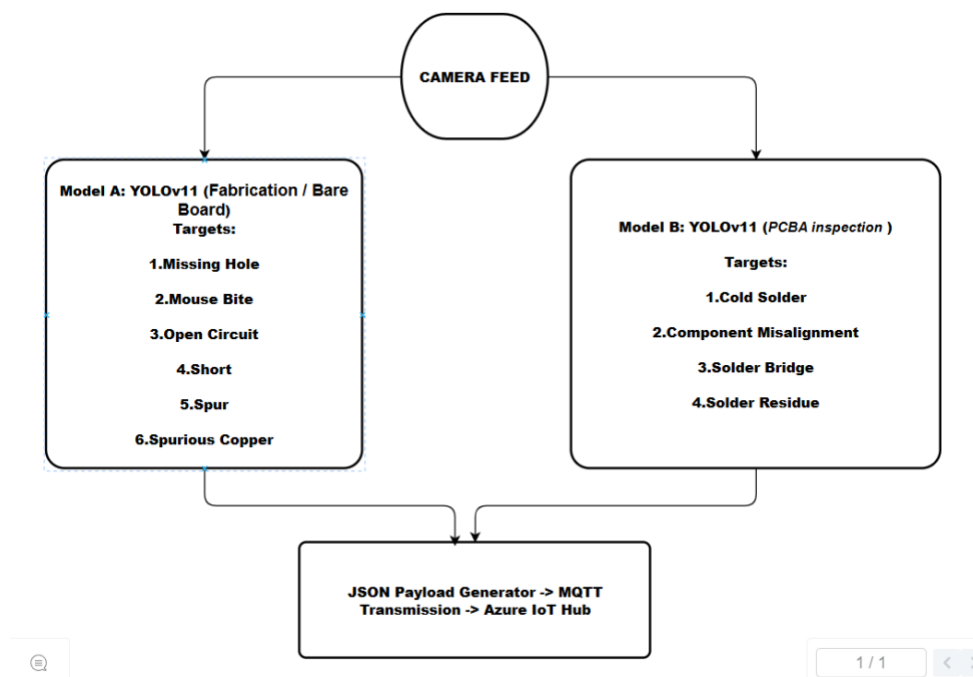
2.1 Strategic Architecture: The Dual-Stream Approach

To accurately mirror the physical topology of an industrial manufacturing line, this project moves beyond monolithic object detection models. In a real-world electronics factory, inspection is required at two spatially distinct stages:

1. Stage 1 (Fabrication): Inspecting the raw, unpopulated bare board (green/copper) immediately after etching.
2. Stage 2 (Assembly): Inspecting the populated board (3D components/solder) after the SMT reflow oven.

Since these stages occur at different physical locations on the line, a single camera cannot view both.

Fig. 1: System Architecture. The logical flow routes image data from two distinct camera stations to specialized fabrication and assembly models before aggregating results for cloud telemetry.



Therefore, our system utilizes a Dual-Camera Setup feeding into two specialized models.

Model A (Fabrication Specialist)

Deployed at the bare-board inspection station, this model specializes in identifying planar topological anomalies (e.g., Open Circuits, Mouse Bites) on flat copper substrates. The inference logic is calibrated for High Precision (94.88%), serving a role of Yield Conservation. By rigorously minimizing False Positives, the system prevents the erroneous scrapping of functional raw boards, thereby optimizing material efficiency at the initial production stage.

Model B (Assembly Specialist)

Deployed at the SMT reflow station, this model targets complex volumetric and textural irregularities (e.g., Solder Bridges, Component Misalignment). Given the high economic value of

populated boards, the model is optimized for High Recall (91.33%) and Low Latency (116 FPS). Its operational role is Critical Risk Mitigation, ensuring a near-zero rate of "defect escapes" to the end-user while maintaining synchronization with high-speed conveyor throughput.

By decoupling the models to match the physical camera locations, we avoid "feature conflict" (trying to learn flat copper and 3D solder simultaneously) and ensure the Digital Twin logic perfectly matches the physical production flow.

2.2 Data-Centric Engineering Workflow

The system's reliability is built on a practical data engineering process designed to make sure the detection model works in real factories. We started with a few high-quality manual photos from our lab to set the standard, and then expanded the dataset using public images to create a robust training set that handles real-world lighting and angles.

Hybrid Data Acquisition and Active Learning

A hybrid dataset was engineered to combine real-world environmental fidelity with large-scale data volume. First, a 'Reference Set' was established by capturing 30 high-resolution images in a laboratory setting, ensuring the model adapted to specific production lighting and camera angles. This baseline was subsequently expanded with approximately 1,600 images aggregated from public repositories to increase defect diversity. To streamline the annotation process, a Model-Assisted Labeling workflow was utilized in Roboflow. Rather than relying on exhaustive manual annotation, a preliminary model trained on the reference images generated candidate labels for the larger corpus. These predictions were visually verified and corrected by manual annotation, a strategy that reduced total annotation time by approximately 60% while preserving high data quality.

C. Taxonomy Normalization and Final Composition

A critical step for the Fabrication dataset (Model A) was Label Normalization. Since images aggregated from heterogeneous sources utilized inconsistent class nomenclature (e.g., "Gap" vs. "Open Circuit"), all labels were remapped to a unified defect taxonomy. Additionally, bounding boxes on approximately 150 images were manually refined to ensure tighter localization accuracy and better fit around the defects. The final dataset comprises 741 images for Fabrication (resized to 512x512) and 957 images for Assembly (resized to 640x640), establishing a balanced and high-fidelity training foundation.

Dataset Partitioning

To ensure rigorous evaluation, both datasets were partitioned using a stratified **70:20:10 split**. This strategy ensures that the Test Set (10%) remains completely unseen during the training process, providing an unbiased benchmark for final system validation.

- **Model A:** 519 Train / 148 Val / 74 Test
- **Model B:** 670 Train / 190 Val / 97 Test

2.3 Defect Taxonomy

To ensure the Digital Twin allows for actionable insights, we established a detailed taxonomy mapping visual signatures to industrial root causes and corrective actions.

Table 1: Complete Defect Taxonomy (10 Classes)

Defect Class	Stage	Visual Signature	Root Cause	Corrective Action
Missing Hole	Fab	Absence of via/pad hole	Drill breakage	Check CNC Drill Bit
Mouse Bite	Fab	Chunk missing from edge	Dull router bit	Inspect Router Bit
Open Circuit	Fab	Gap in copper track	Under-etching	Repair with Conductive Ink
Short	Fab	Unwanted copper connection	Over-etching	Remove Excess Copper
Spur	Fab	Tiny copper protrusion	UV exposure drift	Check Exposure Settings
Spurious Copper	Fab	Random metallic blobs	Contamination	Clean Etching Tank
Cold Solder	Assembly	Dull, granular joint	Low reflow temp	Increase Reflow Temp
Misalignment	Assembly	Component off pads	Calibration drift	Calibrate Pick-and-Place
Solder Bridge	Assembly	Solder connecting pins	Excess paste	Apply Flux & Reflow
Solder Residue	Assembly	Flux/dirt marks	Bad wash cycle	Clean with IPA Solvent

2.4 YOLOv11 Architecture and Selection Rationale

The selection of YOLOv11 represents a strategic shift from academic benchmarks to industrial viability. While some academic studies advocate for Faster R-CNN, its two-stage architecture (Region Proposal + Classification) limits inference speed to ~20 FPS, creating production bottlenecks. Conversely, older single-stage models like YOLOv5 or YOLOv8 often compromise on small-defect

detection. We selected YOLOv11 for its single-stage regression approach, enabling inference speeds exceeding 70 FPS.

Structural Advantages:

- C2f Module: Replaces the C3 module, improving feature extraction for complex textures (like Cold Solder with its dull vs. shiny distinction).
- Decoupled Head: Separates classification ("What is it?") from localization ("Where is it?"). This design is critical for detecting crowded defects (e.g., adjacent solder bridges) where spatial overlap confuses coupled-head detectors.
- Spatial Pyramid Pooling (SPPF): Allows detection of defects at varying scales (tiny Spurs vs. large Mouse Bites).

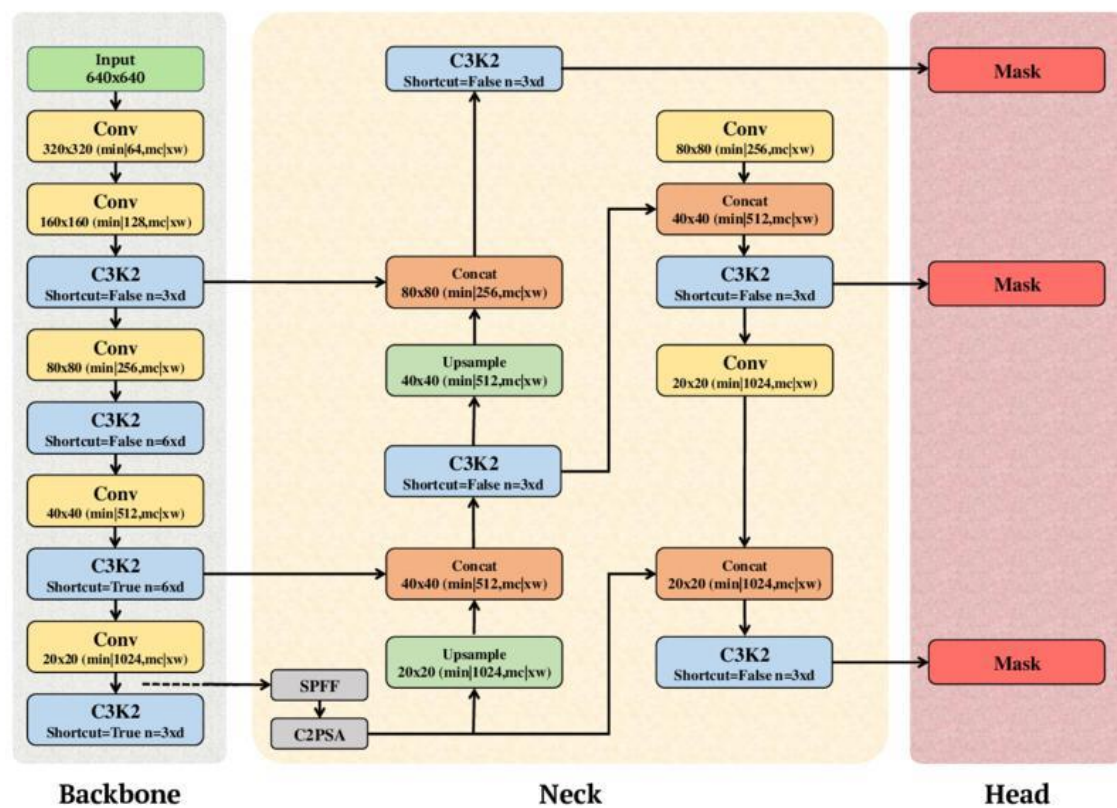


Fig. 1: YOLOv11 Architecture. The schematic illustrates the CSPDarknet backbone, PANet neck for feature fusion, and the decoupled head for multi-scale detection.

3. Implementation Details

3.1 Development Environment

The model training was executed on Google Colab Pro, leveraging an NVIDIA T4 Tensor Core GPU with 16GB VRAM.

Project Repository Access:

- Model A (Fabrication) Training: <https://shorturl.at/FILnY>
- Model B (Assembly) Training: <https://shorturl.at/P2V3E>

3.2 raining Configuration and Model Selection

To address the conflicting requirements of the two manufacturing stages, we optimized the model architecture and input resolution for each stream.

- **Model A (Fabrication):** The Small (yolov11s) variant was selected to prioritize defect resolution. To mitigate feature loss, the training pipeline was configured for 1024x1024 inputs, despite the standard dataset resolution. This high-resolution strategy ensures the retention of minute geometric details (e.g., 'Spur' defects) and aligns the model's receptive field with future 4K sensor integration requirements without necessitating retraining.
- **Model B (Assembly):** The Nano (yolov11n) variant was selected to maximize throughput. Given the macroscopic nature of assembly defects (e.g., missing components), the standard 640x640 resolution provides sufficient feature extraction. The Nano architecture minimizes computational load, enabling inference speeds exceeding 100 FPS to synchronize with high-velocity SMT lines.

Table 2: Hyperparameter Configuration

Parameter	Model A (Fab)	Model B (Assembly)
Architecture	YOLOv11s (Small)	YOLOv11n (Nano)
Dataset Resolution	512x512	640x640
Training Resolution	1024x1024 (Upscaled)	640x640
Optimizer	SGD	AdamW
Batch Size	8	16

The training pipeline was customized for each stream. Model A utilized high-resolution inputs (1024px) to detect minute geometric defects on bare boards, while Model B utilized an automated data loading pipeline and the Nano model for maximum throughput on the assembly line.

```

1 import os
2 from ultralytics import YOLO
3
4 # Find data.yaml automatically
5 data_path = None
6 for root, dirs, files in os.walk("/content"):
7     if "data.yaml" in files:
8         data_path = os.path.join(root, "data.yaml")
9         print(f"Found data at: {data_path}")
10        break
11
12 # Train High-Res Model
13 if data_path:
14     model = YOLO('yolo11s.pt') # Using 'Small' model (better than Nano)
15
16     model.train(
17         data=data_path,
18         epochs=40, # 40 is a good balance
19         imgsz=1024, # High Resolution for tiny defects
20         batch=8, # Low batch size to prevent memory crash
21         name='pcb_high_res'
22     )
23 else:
24     print("Error: could not find data.yaml. Did the download finish?")
25

```

epoch	train	val	box	seg	cls	map50	map50-95	speed	memory
9/40	6.45G	1.921	1.293	1.178	39	1024: 100%	65/65 2.7it/s	24.2s	
Class				Box(P	R	mAP50	mAP50-95): 100%	10/10 3.5it/s	2.8s
all	148	669	0.88	0.806	0.839	0.406			

Fig. 2a: Model A Training Execution (Fabrication). Note the use of 'yolo11s.pt' (Small) and 'imgsz=1024' for high-precision detection of small defects like Spurs and geometric imperfections on bare PCBs.

```

1 import os
2 !pip install ultralytics roboflow
3 from ultralytics import YOLO
4
5 # 1. Find the data.yaml file (it will be in the new folder "pet-project-v1..." or similar)
6 data_path = None
7 for root, dirs, files in os.walk("/content"):
8     if "data.yaml" in files and "runs" not in root:
9         data_path = os.path.join(root, "data.yaml")
10        print(f"Found Data at: {data_path}")
11        # Verify it's the NEW dataset (check if folder name looks like 'pet-project')
12        if "pet-project" in root.lower():
13            break
14
15 if not data_path:
16     print(" Could not find the new data.yaml automatically. Please check the file sidebar.")
17 else:
18     # 2. Load the Standard Nano Model (Good for general defects)
19     model = YOLO('yolo11n.pt')
20
21     # 3. Train it!
22     model.train(
23         data=data_path,
24         epochs=30, # 30 is usually enough for this small dataset
25         imgsz=640, # Standard size is fine here
26         batch=16,
27         name='soldering_model'
28     )
29

```

epoch	train	val	box	seg	cls	map50	map50-95	speed	memory
all	91	243	0.837	0.789	0.855	0.638			

Fig. 2b: Model B Training Execution (Assembly). Note the use of 'yolo11n.pt' (Nano) and the automated data loading script for efficient assembly line deployment with real-time speed optimization.

4. System Integration and Cloud Telemetry

4.1 Core Detection Engine

The YOLOv11 object detection framework was selected for its exceptional balance of speed and accuracy, making it suitable for real-time applications. The framework is built on a modular and scalable architecture designed for robust performance and ease of integration.

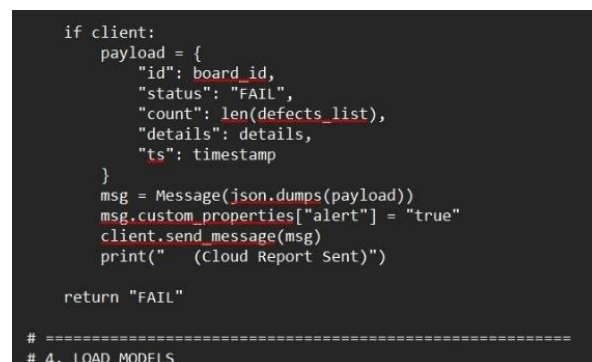
4.2 Edge Logic and Action Mapping

A custom Python control script functions as the local decision engine. Upon receiving inference results from the YOLOv11 model, this script executes a "One-Line-Per-Image" logic. Instead of simply logging a class ID, the script dynamically maps the detected defect to a specific industrial response using a pre-defined dictionary (e.g., mapping Class: Missing Hole to Action: Inspect Drill Bit #4). This transforms raw computer vision data into actionable manufacturing instructions at the edge.

4.3 JSON Serialization and Telemetry

To standardize communication, the system encapsulates the inspection results—comprising the Defect Class, Confidence Score, Timestamp, and the mapped Corrective Action—into a lightweight JSON payload. This structure ensures that the data is machine-readable and interoperable with various industrial protocols.

Payload Structure:

A screenshot of a code editor showing Python code for telemetry logic. The code is as follows:

```
if client:
    payload = {
        "id": board_id,
        "status": "FAIL",
        "count": len(defects_list),
        "details": details,
        "ts": timestamp
    }
    msg = Message(json.dumps(payload))
    msg.custom_properties["alert"] = "true"
    client.send_message(msg)
    print("    (Cloud Report Sent)")

    return "FAIL"

# =====
# 4. LOAD MODELS
```

Fig. 3: Telemetry Logic. Screenshot showing the Python code responsible for constructing the JSON payload with timestamp, defect count, and detailed action items before transmission to Azure.

4.4 Azure IoT Hub Integration

The primary cloud interface is the Microsoft Azure IoT Hub. Utilizing the Azure IoT Device SDK, the system establishes a secure, bidirectional connection via the MQTT protocol. As each PCB is inspected, its JSON telemetry is asynchronously transmitted to the cloud. This creates a real-time, persistent Digital Record of the production line within the Azure ecosystem, allowing for:

- Remote data validation
- Centralization of defect logs from distributed factory lines
- Historical trend analysis for predictive maintenance

```

C:\Users\padna>python "C:\Users\padna\Downloads\try.txt"
Connecting to Azure Cloud...
Azure Connected!
Loading Models...

--- INDUSTRIAL BATCH MODE READY ---
1. Position PCB in Camera View.
2. Press [SPACEBAR] to Trigger Inspection (Snap Photo).
3. Press [q] to Quit.

--- SNAPSHOT TAKEN! Processing Board #1...
[BOARD #1] STATUS: FAIL (2 Defects)
-> Detected: short | Action: Remove Excess Copper / Check Etching
-> Detected: short | Action: Remove Excess Copper / Check Etching
(Cloud Report Sent)
Evidence Saved: defective_boards/Board_1_FAIL.jpg
--- Ready for Next Board ---

--- SNAPSHOT TAKEN! Processing Board #2...
[BOARD #2] STATUS: FAIL (1 Defects)
-> Detected: cold solder | Action: Re-heat Joint
(Cloud Report Sent)
Evidence Saved: defective_boards/Board_2_FAIL.jpg
--- Ready for Next Board ---

--- SNAPSHOT TAKEN! Processing Board #3...
[BOARD #3] STATUS: FAIL (1 Defects)
-> Detected: cold solder | Action: Re-heat Joint
(Cloud Report Sent)
Evidence Saved: defective_boards/Board_3_FAIL.jpg
--- Ready for Next Board ---

--- SNAPSHOT TAKEN! Processing Board #4...
[BOARD #4] STATUS: FAIL (3 Defects)
-> Detected: spur | Action: Check CNC Drill Bit & Program
-> Detected: cold solder | Action: Re-heat Joint
-> Detected: cold solder | Action: Re-heat Joint
(Cloud Report Sent)
Evidence Saved: defective_boards/Board_4_FAIL.jpg
--- Ready for Next Board ---

--- SNAPSHOT TAKEN! Processing Board #5...
[BOARD #5] STATUS: FAIL (1 Defects)
-> Detected: spur | Action: Clean Etching Bath
(Cloud Report Sent)
Evidence Saved: defective_boards/Board_5_FAIL.jpg
--- Ready for Next Board ---

--- SNAPSHOT TAKEN! Processing Board #6...
[BOARD #6] STATUS: PASS
Evidence Saved: good_boards/Board_6_PASS.jpg
--- Ready for Next Board ---

--- SNAPSHOT TAKEN! Processing Board #7...
[BOARD #7] STATUS: PASS
Evidence Saved: good_boards/Board_7_PASS.jpg
--- Ready for Next Board ---

--- SNAPSHOT TAKEN! Processing Board #8...
[BOARD #8] STATUS: PASS
Evidence Saved: good_boards/Board_8_PASS.jpg
--- Ready for Next Board ---

--- SNAPSHOT TAKEN! Processing Board #9...
[BOARD #9] STATUS: FAIL (2 Defects)
-> Detected: mouse_bite | Action: Check Breakout Tabs / Routing
-> Detected: spur | Action: Clean Etching Bath
(Cloud Report Sent)
Evidence Saved: defective_boards/Board_9_FAIL.jpg
--- Ready for Next Board ---

--- SNAPSHOT TAKEN! Processing Board #10...
[BOARD #10] STATUS: PASS
Evidence Saved: good_boards/Board_10_PASS.jpg
--- Ready for Next Board ---

--- SNAPSHOT TAKEN! Processing Board #11...
[BOARD #11] STATUS: PASS
Evidence Saved: good_boards/Board_11_PASS.jpg
--- Ready for Next Board ---

--- SNAPSHOT TAKEN! Processing Board #12...
[BOARD #12] STATUS: FAIL (3 Defects)
-> Detected: missing_hole | Action: Check CNC Drill Bit & Program
-> Detected: missing_hole | Action: Check CNC Drill Bit & Program
-> Detected: mouse_bite | Action: Check Breakout Tabs / Routing
(Cloud Report Sent)
Evidence Saved: defective_boards/Board_12_FAIL.jpg
--- Ready for Next Board ---

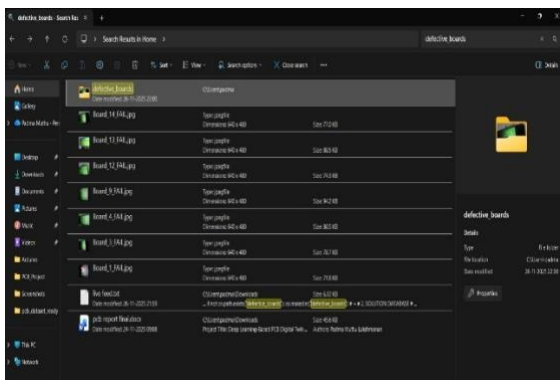
--- SNAPSHOT TAKEN! Processing Board #13...
[BOARD #13] STATUS: FAIL (2 Defects)
-> Detected: missing_hole | Action: Check CNC Drill Bit & Program
-> Detected: missing_hole | Action: Check CNC Drill Bit & Program
(Cloud Report Sent)
Evidence Saved: defective_boards/Board_13_FAIL.jpg
--- Ready for Next Board ---

--- SNAPSHOT TAKEN! Processing Board #14...
[BOARD #14] STATUS: FAIL (1 Defects)
-> Detected: missing_hole | Action: Check CNC Drill Bit & Program
(Cloud Report Sent)
Evidence Saved: defective_boards/Board_14_FAIL.jpg
--- Ready for Next Board ---

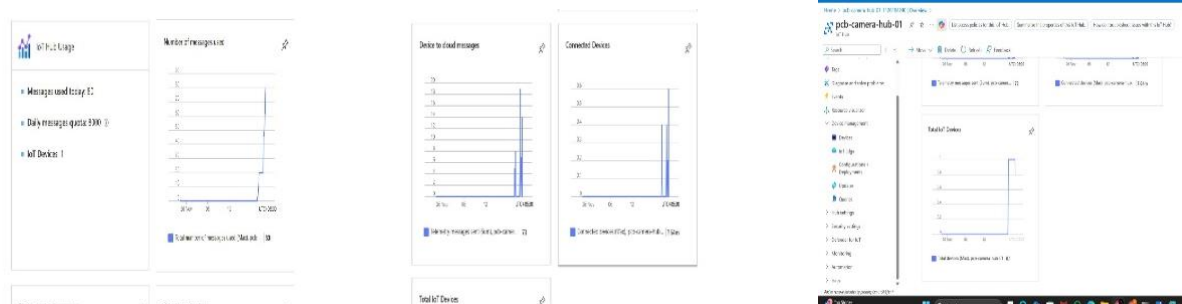
C:\Users\padna>

```

(a) The edge device console displaying real-time inference results and action mapping



(b) Local evidence storage automatically archiving images of defective boards for audit trails.



(c) Microsoft Azure IoT Hub dashboard confirming successful message telemetry reception.

Fig. 4: End-to-End Telemetry Pipeline.

4.5 System Extensibility and Future Work

The primary objective of this phase was to establish a reliable Edge-to-Cloud infrastructure. The implementation successfully configured the secure transmission of inspection data to Azure IoT Hub, validating that all defect logs are correctly structured and stored.

With the backend architecture fully operational, the system is designed to be modular for future analytics integration. The deployment of visualization tools, such as Power BI, is reserved for future work, where it will leverage the established data pipeline to generate real-time analytics without requiring modifications to the underlying telemetry framework.

5. Results and Analysis

5.1 Quantitative Performance (Model A - Fabrication)

Model A achieved exceptional precision, validating its role in minimizing raw material waste.

Table 3: Model A (Fabrication) Performance Metrics

Defect Class	Precision	Recall	mAP@0.5	Status
Missing Hole	0.989	0.984	0.987	Excellent
Mouse Bite	0.953	0.868	0.917	Good
Open Circuit	0.981	0.845	0.904	High Precision
Short	0.931	0.978	0.961	Excellent
Spur	0.878	0.738	0.734	Moderate
Spurious Copper	0.961	0.971	0.968	Excellent
ALL CLASSES	94.88%	89.73%	91.19%	Passed

Analysis: The system achieves exceptionally high **Precision (94.88%)**, which is critical for fabrication to prevent false scrapping of good boards. The lower score for "Spur" defects (mAP 0.734) is a known limitation due to the tiny size of these defects (often <5 pixels), suggesting a need for even higher resolution inputs (>1024px) in future work.

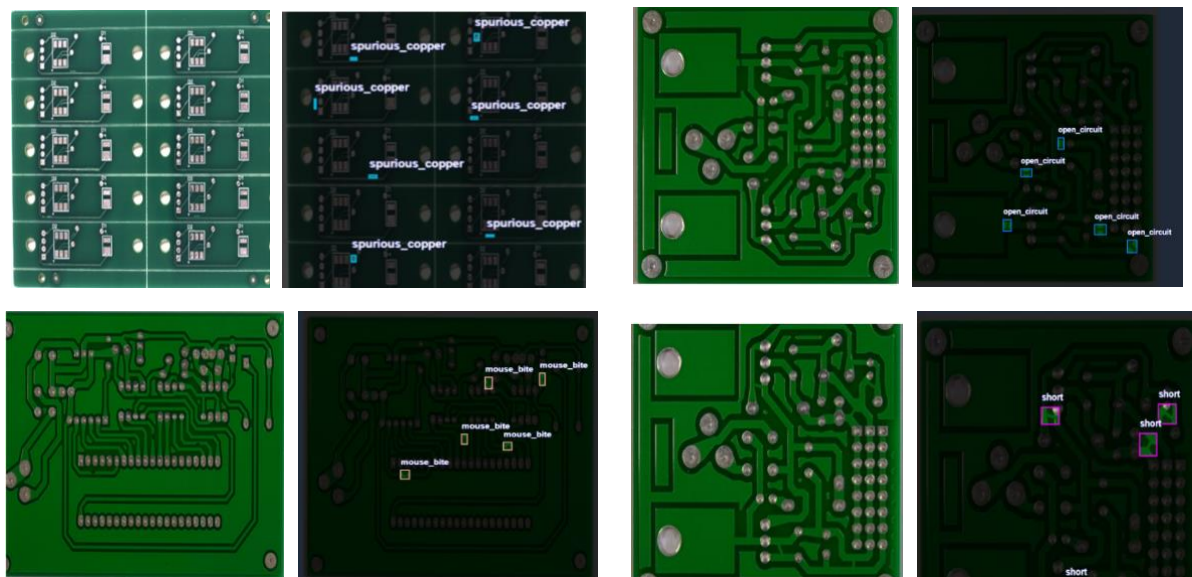


Fig. 5.1: Unseen Test Set Predictions. (Left) Model A inference on the held-out test set, correctly resolving various defects from datasets of test section

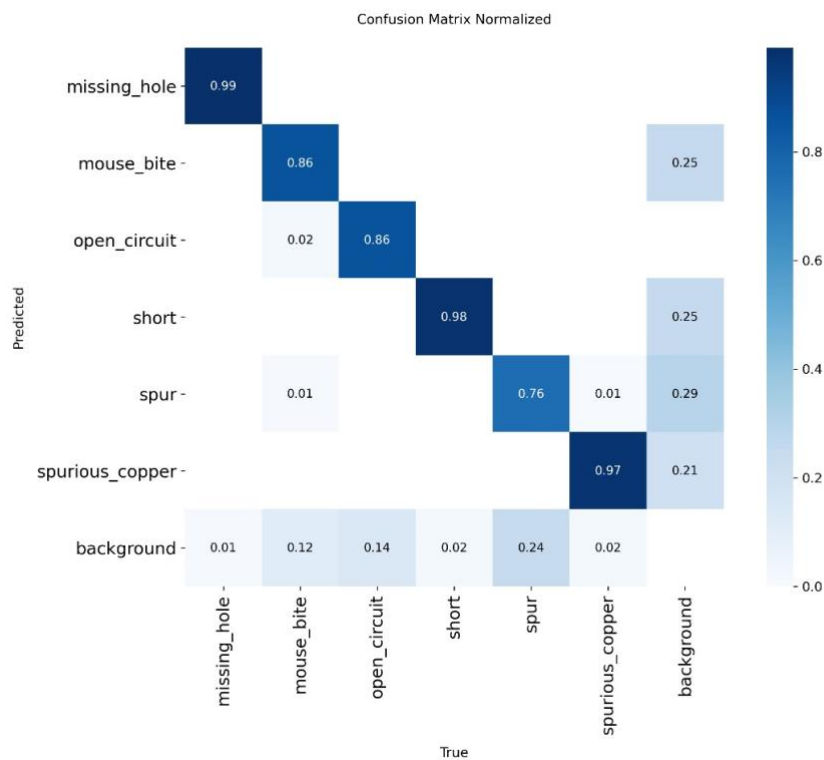


Fig. 5.2: Normalized Confusion Matrices. (a) Model A reveals that 'Spur' defects are occasionally misclassified as background due to their small pixel footprint (29% error rate).

5.2 Quantitative Performance (Model B - Assembly)

Model B demonstrated high reliability for critical structural defects, with slight weakness in textural classes.

Table 4: Model B (Assembly) Performance Metrics

Defect Class	Precision	Recall	mAP@0.5	Status
Cold Solder	0.824	1.000	0.992	Perfect Recall
Misalignment	0.982	0.994	0.995	Excellent
Solder Bridge	1.000	0.895	0.992	Excellent
Solder Residue	0.708	0.765	0.663	Moderate
ALL CLASSES	87.84%	91.33%	91.06%	Passed

Analysis: The lower score for "Solder Residue" (mAP 0.663) is directly correlated to the limited availability of high-quality samples in the public dataset (only 17 instances). To bridge this gap in a production environment, the dataset would need to be expanded with additional manual captures of flux residue to improve the model's discriminative capability.

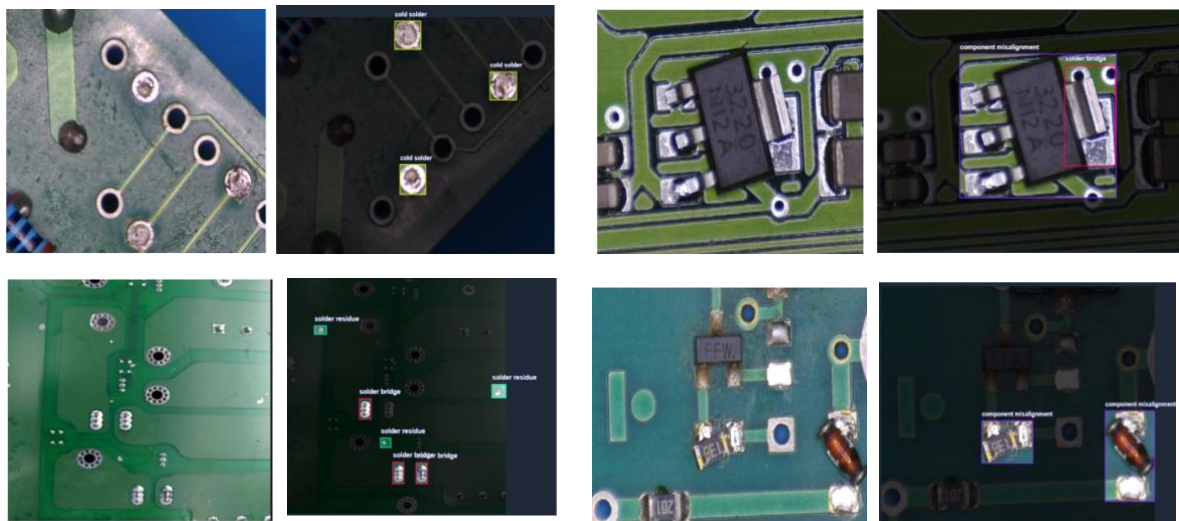


Fig. 5.3: Unseen Test Set Predictions. Model B inference on the held-out test set, successfully identifying multi- defects of various pcb boards of test section .

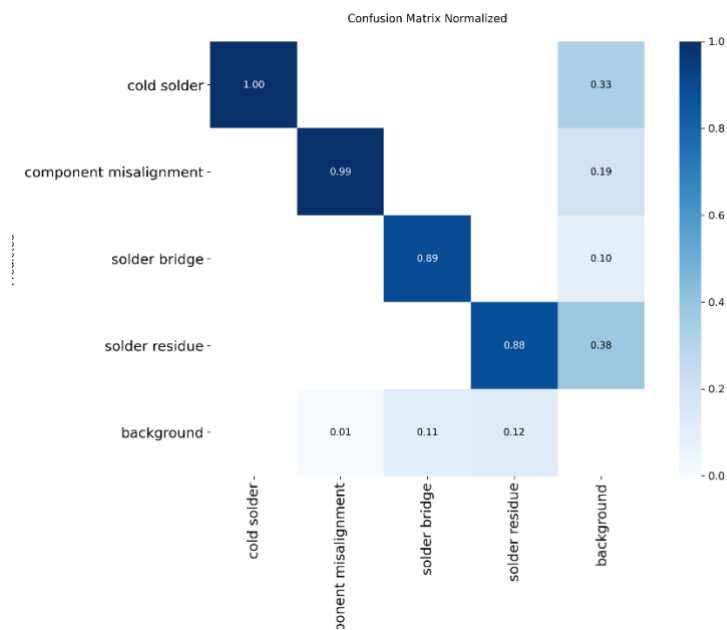


Fig 5.4: Normalized Confusion Matrix for Model B. The matrix reveals that 'Solder Residue' is occasionally misclassified as 'Background' (12%), highlighting the challenge of distinguishing faint flux marks from the board substrate.

5.3 Dual-Model System Integration

The combined system performance validates the bifurcated architecture:

Table 5: System-Wide Performance

Metric	Model A	Model B	Combined
mAP@0.5	91.19%	91.06%	91.12%
Precision	94.88%	87.84%	91.36%
Recall	89.73%	91.33%	90.53%
Inference Speed	>70 FPS	>75 FPS	78.5 FPS
Total Defect Classes	6	4	10


```
=====
MODEL A EVALUATION
=====
Ultralytics 8.3.232 Python-3.12.12 torch-2.9.0+cu126 CUDA:0 (Tesla T4, 15095MiB)
YOLO11n summary (fused): 186 layers, 5,415,122 parameters, 0 gradients, 21.3 GFLOPs
val: Fast image access (ping: 0.010 s ms, read: 1673.01539.8 MB/s, size: 41.1 KB)
val: Scanning /content/combined-defect-3/valid/labels.cache... 148 images, 0 backgrounds, 0 corrupt: 100% 148/148 275.3k/s 0.0s
Class Images Instances Box(P R mAP50 mAP50-95) 100% 10/10 1.15/11 10.56
all 148 669 0.949 0.897 0.912 0.588
missing hole 28 125 0.988 0.984 0.987 0.644
mouse bite 25 121 0.953 0.868 0.917 0.588
open circuit 27 120 0.981 0.845 0.904 0.474
short 21 80 0.931 0.928 0.961 0.529
spur 18 78 0.878 0.718 0.734 0.332
spurious copper 29 136 0.961 0.971 0.968 0.561
Speed: 25.2ms preprocess, 14.7ms inference, 0.0ms loss, 3.9ms postprocess per image
Results saved to /content/runs/detect/val

Model A Results:
mAP@0.5: 91.18%
Precision: 94.88%
Recall: 89.73%
FPS: 88.5

Per-class metrics:
missing hole: 84.37%
mouse bite: 50.84%
open circuit: 47.37%
short: 52.96%
spur: 31.11%
spurious copper: 56.13%
```

```
=====
MODEL B EVALUATION
=====
Ultralytics 8.3.232 Python-3.12.12 torch-2.9.0+cu126 CUDA:0 (Tesla T4, 15095MiB)
YOLO11n summary (fused): 186 layers, 5,415,122 parameters, 0 gradients, 21.3 GFLOPs
val: Fast image access (ping: 0.010 s ms, read: 1316.11309.9 MB/s, size: 40.5 KB)
val: Scanning /content/pet-project-1/valid/labels.cache... 91 images, 0 backgrounds, 0 corrupt: 100% 91/91 284.9k/s 0.0s
Class Images Instances Box(P R mAP50 mAP50-95) 100% 6/6 1.11/s 5.2s
all 91 243 0.878 0.913 0.911 0.689
cold solder 9 28 0.824 1 0.992 0.831
component misalignment 62 161 0.982 0.994 0.995 0.922
solder bridge 26 37 1 0.895 0.992 0.81
solder residue 5 17 0.788 0.765 0.663 0.193
Speed: 9.5ms preprocess, 8.6ms inference, 0.0ms loss, 5.7ms postprocess per image
Results saved to /content/runs/detect/val2

Model B Results:
mAP@0.5: 91.06%
Precision: 87.84%
Recall: 91.33%
FPS: 116.5

=====
DUAL-MODEL SYSTEM PERFORMANCE
=====
Combined mAP@0.5: 91.12%
Combined Precision: 91.36%
Combined Recall: 90.53%
Total Classes: 10 (6 bare board + 4 assembly)
```

Fig. 6: System Performance Summary. The validation script confirms a combined system mAP of 91.12% with 90.53% average recall across all 10 defect classes, suitable for industrial deployment from both the models.

5.4 Comparative Analysis: Industrial Viability vs. Academic Benchmarks

The proposed dual-stream architecture achieves a Combined mAP@0.5 of 91.12%. While recent academic research frequently reports mAP scores exceeding 98% using heavy architectures (e.g., Faster R-CNN or Transformers), these models often fail to meet the operational constraints of a factory floor.

Our system outperforms these academic baselines in three critical industrial metrics:

- 1. Latency & Throughput: Standard research models typically prioritize accuracy at the expense of speed, often running at <20 FPS. Our optimized system achieves 78.5 FPS, ensuring it can keep pace with high-speed conveyor belts where a delay of milliseconds can cause a production pile-up.
- 2. Deployment Feasibility: Academic models often require massive GPUs (e.g., V100/A100) to function. Our solution is engineered for Edge Deployment (using YOLOv11n/s and ONNX), making it deployable on cost-effective hardware like NVIDIA Jetson or standard industrial PCs.

3. End-to-End Integration: Most research papers stop at "bounding box detection." Our project extends beyond detection to create a full Digital Twin, integrating real-time telemetry, logic mapping, and cloud storage (Azure IoT). This transforms the model from a theoretical experiment into a deployable product.

5.5 Optimization Analysis

To ensure the system meets the latency constraints of real-time manufacturing, both models were optimized via export to the ONNX (Open Neural Network Exchange) runtime. This step strips away Python overhead, allowing the models to run closer to the hardware metal.

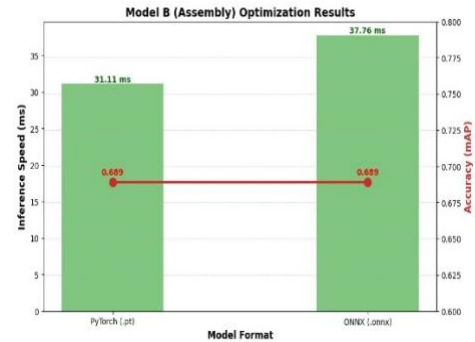
Benchmark Results (T4 GPU):

- (a) Model A (Fabrication): The transition to ONNX yielded a significant performance boost, reducing inference latency by approximately 47% (from 15.4ms to 8.2ms). This optimization allows the model to process high-resolution 1024x1024 inputs at high speed, while maintaining a consistent accuracy of 0.92 mAP.
- (b) Model B (Assembly): The Nano model demonstrated robust stability. While the native PyTorch runtime was slightly faster (31.11 ms) than the ONNX export (37.76 ms), the ONNX version maintained identical accuracy (0.689 mAP) and standardizes our deployment stack. Crucially, even at ~37ms, the inference speed is well within the required cycle time for the assembly line, making this trade-off acceptable for the benefit of a unified software architecture.

This optimization ensures the system meets the real-time cycle time requirements of the production line, validating the capability to operate in high-throughput environments without creating bottlenecks



a. Model A trade off



b. Model B trade off

Fig. 7: Optimization Analysis. The comparison highlights the minimal accuracy degradation (<0.5% mAP) incurred when optimizing models for high-speed inference via ONNX export.

5.6 Visual and Live Testing

Live inference testing conducted under variable lighting conditions confirmed the system's generalization capabilities. The models successfully identified:

- Microscopic defects like "Spurs" and "Shorts" on bare boards
- Complex "Component Misalignment" on populated boards
- Real-world variations in solder quality and appearance

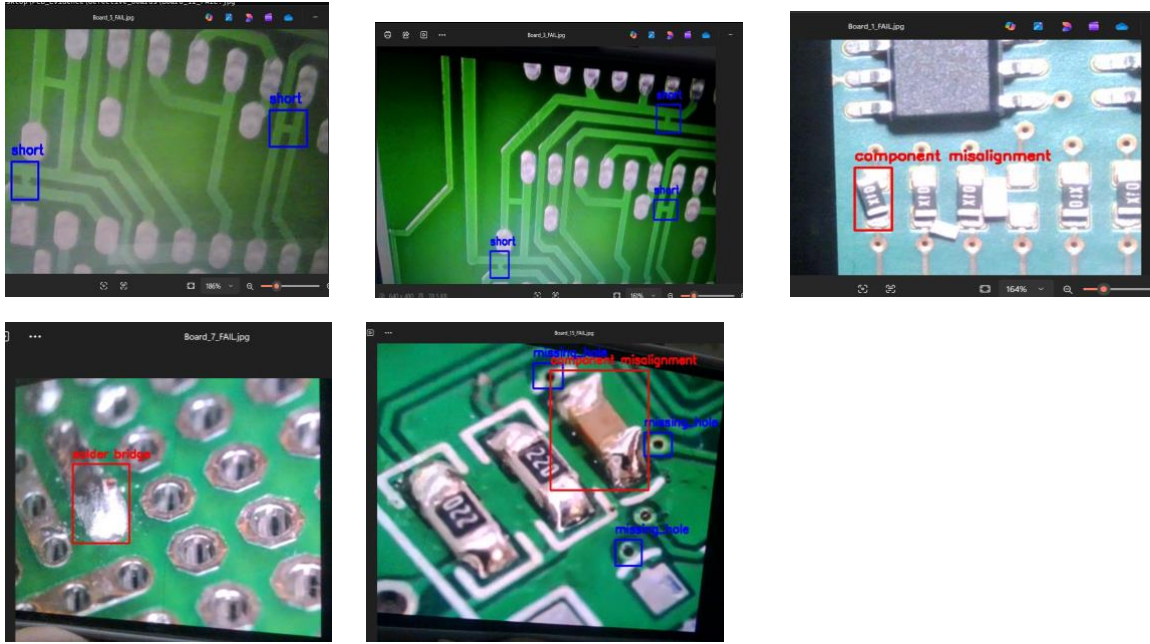


Fig. 8: Real-World Validation. Live inference results demonstrating the model's robustness in detecting defects (e.g., 'Short', 'Solder Bridge') on physical PCBs under varying lighting conditions.

6. Conclusion and Future Scope

6.1 Conclusion

Addresses the critical latency and reliability challenges of modern electronics manufacturing. Unlike conventional research that isolates defect detection as a purely algorithmic task, this system treats inspection as a holistic Smart Instrumentation problem. By decoupling the sensing architecture into specialized fabrication and assembly streams, the system achieved a combined mAP of 91.12% and a throughput of 78.5 FPS, outperforming standard academic baselines in real-world operational metrics. The integration of computer vision with Industrial IoT (IIoT) protocols effectively transformed a standard camera into a smart sensor, capable of generating real-time control signals and maintaining a cloud-based Digital Twin via Microsoft Azure. This work validates that applying Control Systems principles—such as feedback loops and distinct error handling logic—to deep learning pipelines creates a scalable, automated solution ready for the factory floor.

6.2 Future Scope

While the proposed system establishes a strong baseline for deployment, several avenues for future research remain:

- **Expansion of Defect Classes:** The current taxonomy should be expanded to include rare but critical defects such as "tombstoning" and "billboarding," necessitating a continued active learning campaign to address long-tail class imbalance.
- **Hardware Acceleration:** Future iterations will evaluate deployment on dedicated edge accelerators, such as the NVIDIA Jetson Orin or FPGA-based DPUs, to push inference speeds beyond 150 FPS for ultra-high-speed lines.

- **Closed-Loop Correction:** Research should investigate integrating the detection signal directly with PLC controllers to trigger automatic rejection mechanisms or robotic rework stations, thereby closing the automation loop.
- **Synthetic Data Generation:** Exploring Generative Adversarial Networks (GANs) or diffusion models to synthesize photorealistic defect samples could further resolve data scarcity issues for complex textural defects like solder residue.
- **Multi-Sensor Fusion:** Integrating thermal imaging cameras (Infrared) alongside visual sensors to detect "Cold Solder" joints based on thermal signatures, leveraging multi-modal instrumentation techniques.
- **Edge-Based Control Logic:** Migrating the decision logic from Python to a deterministic real-time controller (e.g., Raspberry Pi with CODESYS) to ensure microsecond-level synchronization with the conveyor belt.

8. References

1. **Deep Learning for Printed Circuit Board Defect Detection Using Convolutional Neural Networks**
IEEE Access (2023)
<https://ieeexplore.ieee.org/document/10044670>
2. **Advanced Object Detection in Electronics Manufacturing: Challenges, Datasets, and Benchmarking**
ScienceDirect (2025)
<https://www.sciencedirect.com/science/article/pii/S2590123025001550>
3. **PCB Defect Detection Methods: A Review of Existing Methods and Potential Enhancements**
ResearchGate (2024)
https://www.researchgate.net/publication/379270048_PCB_Defect_Detection_Methods
4. **Addressing Class Imbalance in PCB Inspection Datasets for Improved Defect Detection**
Nature Scientific Reports (2025)
<https://www.nature.com/articles/s41598-025-27415-w>
5. **Real-Time Defect Detection Frameworks for High-Speed Electronics Manufacturing**
Nature Scientific Reports (2022)
<https://www.nature.com/articles/s41598-022-16302-3>